

**DUY TAN UNIVERSITY
INTERNATIONAL SCHOOL**



GROUP PROJECT

SUBJECT: SOFTWARE MEASURE

TEAM MEMBERS

NGUYEN VAN DUY MANH

LE MINH TRUONG KHA

TRAN HOAI NAM

Da Nang, April 27, 2026

TABLE OF CONTENTS

I. Introduction.....	1
1.1. Purpose	1
1.2. Scope.....	1
1.3. Definitions, Acronyms, Abbreviations	1
1.4. References	2
1.5. Overview	2
II. Overall Description.....	2
2.1. Product Perspective	2
2.2. Product Functions	2
2.3. User Classes and Characteristics	2
2.4. Operating Environment	3
2.5. Design and Implementation Constraints	3
2.6. Assumptions and Dependencies	3
2.7. System States	3
2.8. Data Flow Description.....	4
III. Functional Requirements.....	4
3.1. FR-01: Upload SRS	4
3.2. FR-02: Text Processing (NLP)	5
3.3. FR-03: Analyze with AI.....	5
3.4. FR-04: Calculate Function Points	9
3.5. FR-05: View Dashboard	9
3.6. FR-06: Edit Result	9
3.7. FR-07: Export Report	9
IV. Non-Functional Requirements.....	10
4.1. Performance Requirements.....	10
4.2. Usability Requirements	10

4.3. Reliability Requirements	10
4.4. Security Requirements.....	11
4.5. Maintainability Requirements	11
4.6. Scalability Requirements	11
V. Use Case Specification	11
5.1. Actors	11
5.2. Use Cases.....	11
VI. UML DIAGRAMS.....	12
6.1. UC-01: Upload SRS	12
6.1.1. Use Case Diagram.....	12
6.1.2. Activity Diagram	12
6.1.3. Sequence Diagram	13
6.2. UC-02: Preprocess Text (NLP).....	13
6.2.1. Use Case Diagram.....	13
6.2.2. Activity Diagram.....	14
6.2.3. Sequence Diagram	14
6.3. UC-03: Analyze with AI	15
6.3.1. Use Case Diagram.....	15
6.3.2. Activity Diagram.....	15
6.3.3. Sequence Diagram	15
6.4. UC-04: Calculate Function Points	16
6.4.1. Use Case Diagram.....	16
6.4.2. Activity Diagram	16
6.4.3. Sequence Diagram	16
6.5. UC-05: View Result.....	17
6.5.1. Use Case Diagram.....	17
6.5.2. Activity Diagram	17

6.5.3. Sequence Diagram	17
6.6. UC-06: Edit Result	18
6.6.1. Use Case Diagram.....	18
6.6.2. Activity Diagram.....	18
6.6.3. Sequence Diagram	18
6.7. UC-07: Export Report	19
6.7.1. Use Case Diagram.....	19
6.7.2. Activity Diagram	19
6.7.3. Sequence Diagram	19
VII. Data Requirements.....	20
7.1. AI Output Data (AI Output JSON).....	20
7.2. Final Estimation Data (Final Output)	20
VIII. Acceptance Criteria	20
IX. Demo Project.....	21

I. Introduction

1.1. Purpose

- This document provides a detailed specification of the functional and non-functional requirements for the "AI-Based Function Point Estimation Tool from SRS Documents" system.
- The purpose of this document is to guide the development team, QA/Testers, and stakeholders in clearly understanding the system's behavior, ensuring the accurate implementation of an automated Function Point estimation solution.

1.2. Scope

- The product is a Web-based application designed to automate the Function Point Analysis (FPA) process from Software Requirements Specification (SRS) documents.
- The system addresses core issues:
 - Manual FP counting is too time-consuming.
 - Human bias leads to inconsistent results.
 - Agile environments require rapid and continuous re-estimation.
- The system will perform the following:
 - Extract functional components (EI, EO, EQ, ILF, EIF).
 - Calculate Function Points.
 - Estimate Effort, Schedule, and Cost.
 - Provide a Dashboard to visualize and edit results.

1.3. Definitions, Acronyms, Abbreviations

- FPA: Function Point Analysis.
- EI: External Input.
- EO: External Output.
- EQ: External Inquiry.
- ILF: Internal Logical File.
- EIF: External Interface File.
- UFC: Unadjusted Function Point Count.
- VAF: Value Adjustment Factor.
- LLM: Large Language Model.
- NLP: Natural Language Processing.

- MVP: Minimum Viable Product.

1.4. References

- IFPUG Function Point Counting Practices Manual.
- LLM API Documentation (Gemini, Groq, OpenAI).
- COCOMO Software Estimation Model.

1.5. Overview

- The document is organized as follows:
 - Section 1: Overall description of the product, users, and constraints.
 - Section 2: Detailed specification of System Features (Functional Requirements).
 - Sections 3 - 6: Non-functional requirements, Use Cases, and UML diagrams (Attached in a separate system design document).
 - Sections 7 - 8: Data requirements and Acceptance Criteria.

II. Overall Description

2.1. Product Perspective

- The system is a standalone Web application, utilizing:
 - Backend: Python (Flask).
 - Frontend: HTML/CSS/JavaScript.
 - AI Integration: LLM API services (Gemini/Groq/OpenAI).
- The system does not train models from scratch but utilizes Prompt Engineering techniques.

2.2. Product Functions

- Upload SRS documents.
- Preprocess text (NLP).
- Analyze with AI.
- Calculate Function Points.
- View results on the Dashboard.
- Edit results (Human-in-the-loop).
- Export reports.

2.3. User Classes and Characteristics

- Primary Users: Project Managers / Measurement Analysts.

- Characteristics: Possess basic software knowledge and understand project management, but are not required to be expert-level FPA counters.

2.4. Operating Environment

- Web Browsers: Latest versions of Chrome, Edge, Safari, Firefox.
- Server: Python 3.x execution environment + Flask framework.
- Network: Requires a continuous Internet connection to execute LLM API calls.

2.5. Design and Implementation Constraints

- Schedule: The MVP must be completed within 7 weeks.
- Response Time: LLM API responses must not exceed 10 seconds per request.
- File Format: Only accepts .pdf and .docx files.
- File Management: Uploaded files are deleted immediately after analysis is complete (no permanent storage on the server).
- Session Management: Analysis data is temporarily stored in server RAM or Local Storage. If the user reloads the page (Refresh/F5), the analysis process will be canceled, and unexported data will be lost.
- API Security: API Keys must strictly be configured via Environment Variables in the Backend; hard-coding them into the source code is strictly prohibited.

2.6. Assumptions and Dependencies

- Data Assumptions: The input SRS document must have a clear text structure (not scanned images) and must not be DRM/Password protected. The language used in the document (English/Vietnamese) must follow standard technical writing styles with minimal slang to ensure maximum AI accuracy.
- Business Assumptions: The Value Adjustment Factor (VAF) is defaulted to 1.0 (average level) if the system is in the MVP phase and does not yet require the user to evaluate the 14 General System Characteristics (GSCs).
- Dependencies: The system is directly dependent on the Rate Limit and Uptime of the LLM API providers (Gemini, Groq). Processing time may fluctuate based on third-party latency.

2.7. System States

- Throughout the lifecycle of an estimation session, the system will transition through 4 core states:

- Idle: The system is ready at the file upload screen. No active session has been triggered.
- Processing: Triggered immediately upon a successful file upload. The system sequentially executes: text extraction $\rightarrow$ API calls $\rightarrow$ calculations. The UI is temporarily locked to prevent spam requests.
- Completed: Data is successfully returned. The Dashboard displays all results. The system is ready to accept editing actions (Edit) or report exporting (Export).
- Failed: The process is interrupted due to invalid file formats, API timeouts, or exceeded Quotas. This state requires the system to clean up garbage data and allow the user to try again (Retry).

2.8. Data Flow Description

- The system's data flow follows a Linear Pipeline model:
 - Upload: The user uploads a .pdf/.docx file via the browser $\rightarrow$ The Backend receives the byte stream and stores it in temporary memory (RAM).
 - NLP Extraction: The extractor (PDFMiner/python-docx) converts the file into Raw Text. The NLP module (NLTK) removes noise, tokenizes, and splits the text into Text Chunks.
 - AI Classification: The Backend sends an HTTP POST Request containing the System Prompt + Text Chunks to the LLM API. The LLM returns a JSON string formatted as: {"EI":x, "EO":y...}.
 - Calculation: The Calculator Module at the Backend receives the JSON, multiplies it by the FPA weight matrix, and adds the VAF to generate Total FP, Effort, and Cost.
 - UI Rendering: The Backend packages all return data to the Frontend via API. The browser renders the Dashboard and draws breakdown charts.

III. Functional Requirements

3.1. FR-01: Upload SRS

- Description: Allows the user to upload project specification documents to the system.
- Pre-condition: The user accesses the system; the system is in the Idle state.
- Processing: Validates the file format (MIME type) and file size. Saves the file to temporary memory.

- Post-condition: The file is temporarily stored; the system transitions to the Processing state.
- Exceptions:
 - File exceeds size limit (e.g., > 10MB) → Cancel upload, display "File too large" error.
 - Invalid format (e.g., uploading .txt, .exe) → Display "Only PDF/DOCX formats are accepted" warning.

3.2. FR-02: Text Processing (NLP)

- Description: Extracts and cleans text data from the file.
- Pre-condition: FR-01 is successfully completed.
- Processing: Removes footers/headers, normalizes text, and chunks the text.
- Post-condition: Generates an array of valid Text Chunks ready to be sent to the AI.
- Exceptions: The PDF file contains only scanned images (Scanned PDF). Halt the process, display "System cannot read document content. Please check the file."

3.3. FR-03: Analyze with AI

- Description: Calls the LLM to classify FPA components.
- Pre-condition: FR-02 is completed; the server has an active Internet connection.
- Processing: Pushes Text Chunks via HTTP Client to the LLM API. Parses the JSON response.

```
SYSTEM_PROMPT = """"You are a certified Function Point Analysis (FPA)
expert applying the IFPUG standard.
```

```
Your task: Analyze the given software requirement text and classify
ONLY explicitly mentioned functional components.
```

```
DEFINITIONS (STRICT)
```

```
- EI (External Input):          User input that CHANGES system data
(add, update, delete). Must involve writing/modifying data in an
internal store.
```

- EO (External Output): Output that retrieves data AND applies processing, calculation, or derivation. Must involve computation beyond simple retrieval.
- EQ (External Inquiry): Simple data retrieval with NO derived data, NO calculations. Read-only display/lookup/search.
- ILF (Internal Logical File): A logical group of data MAINTAINED (created/updated/deleted) by this system. Must be an internal data store.
- EIF (External Interface File): Data referenced/read by this system but MAINTAINED by another external system. Must be an external data source.

ABSOLUTE RULES (NO EXCEPTIONS)

1. DO NOT GUESS – Do NOT assume or infer features not explicitly described in the text.
2. EVIDENCE REQUIRED – Each item MUST map to a specific sentence or phrase from the document. If no evidence exists → DO NOT include the item.
3. NO AUTO-GENERATION – Do NOT add items just because "systems usually have them". Only extract what is CLEARLY described.
4. ZERO HALLUCINATION – If unsure about classification → DO NOT include. Missing is better than wrong.
5. NO DUPLICATES – Each distinct functional process must be counted exactly once.
6. CLASSIFY ONLY IF CLEAR – Only classify if the role (input/output/query/store) is UNAMBIGUOUS in the text.

OUTPUT FORMAT (JSON ONLY)

Return ONLY a valid JSON object – no markdown, no explanation, no preamble.

```
{
  "EI": [{"name": "descriptive name", "evidence": "exact quote from
document"}],
  "EO": [{"name": "descriptive name", "evidence": "exact quote from
document"}],
  "EQ": [{"name": "descriptive name", "evidence": "exact quote from
document"}],
  "ILF": [{"name": "descriptive name", "evidence": "exact quote from
document"}],
  "EIF": [{"name": "descriptive name", "evidence": "exact quote from
document"}]
}
```

IMPORTANT RULES FOR OUTPUT:

- If a category has NO items → return an empty array []
- The "evidence" field MUST contain the actual sentence or phrase from the input text that justifies the classification
- The "name" field should be a short descriptive label
- Your ENTIRE response must be ONLY this JSON object, nothing else

FEW-SHOT EXAMPLES

Example 1 – Input text:

"Users submit a registration form with name, email, password. Data is saved to the Users table."

Output:

```
{"EI": [{"name": "User Registration", "evidence": "Users submit a
registration form with name, email, password"}], "EO": [], "EQ": [],
"ILF": [{"name": "Users Table", "evidence": "Data is saved to the
Users table"}], "EIF": []}
```

Example 2 – Input text:

"The admin dashboard shows total revenue calculated from all orders."

Output:

```
{"EI": [], "EO": [{"name": "Revenue Dashboard", "evidence": "admin dashboard shows total revenue calculated from all orders"}], "EQ": [], "ILF": [], "EIF": []}
```

Example 3 – Input text (NO queries mentioned):

"Teachers enter grades. The system stores grades in the database."

Output:

```
{"EI": [{"name": "Enter Grades", "evidence": "Teachers enter grades"}], "EO": [], "EQ": [], "ILF": [{"name": "Grades Database", "evidence": "system stores grades in the database"}], "EIF": []}
```

Note: EQ is [] because NO query/search/lookup is mentioned. Do NOT add EQ just because "users might view grades".

FINAL VALIDATION (MANDATORY – do this before returning)

1. Remove any item WITHOUT a real evidence quote from the input text
2. Remove any item that was ASSUMED or INFERRED
3. Ensure every "evidence" field contains text that actually appears in the input
4. If a category has no items → return []

Now analyze the following requirement text and return ONLY the JSON: ""

- Post-condition: Receives valid count data for EI, EO, EQ, ILF, EIF metrics.
- Exceptions:
 - API Timeout (>10s) or HTTP 500 → Trigger automatic Retry mechanism (up to 3 times).

- API Quota exceeded (HTTP 429) → Transition state to Failed, display "System overloaded, please try again later."

3.4. FR-04: Calculate Function Points

- Description: Computes UFC, VAF, Effort, and Cost based on the component counts.
- Pre-condition: FR-03 returns properly formatted JSON, or the user submits new data from FR-06.
- Processing: Multiplies by coefficients, performs conversion calculations.
- Post-condition: Generates the Final Result Object.
- Exceptions: Invalid JSON format from the AI (e.g., returning a string instead of an int) → Backend automatically validates and parses the error before calculation to prevent a system crash.
- $UFC = \sum (Count \times Weight)$, $FP = UFC \times VAF$

3.5. FR-05: View Dashboard

- Description: Renders the results on the user interface.
- Pre-condition: FR-04 is completed.
- Processing: Populates data into UI components (Charts, Information Cards, Tables).
- Post-condition: The system transitions to the Completed state.
- Exceptions: Empty data => Display No Data screen and a button to return to the homepage.

3.6. FR-06: Edit Result

- Description: Human-in-the-loop mechanism allowing the user to modify FPA values.
- Pre-condition: The system is in the Completed state.
- Processing: Receives changes from the input fields. Sends the update payload to the Backend. Re-triggers FR-04.
- Post-condition: The Dashboard screen and charts are updated immediately based on the new figures.
- Exceptions: The user enters negative numbers or letters into the count fields => UI blocks the input, displays "Only positive integers are accepted" error.

3.7. FR-07: Export Report

- Description: Packages the results into a file for sharing/storage.
- Pre-condition: The system is in the Completed state.

- Processing: Aggregates displayed metrics, generates a CSV/PDF file on the Backend, and sends the file stream to the Client.
- Post-condition: The file is successfully downloaded to the user's browser.
- Exceptions: Connection lost during file download => Display "Download failed" notification.

IV. Non-Functional Requirements

4.1. Performance Requirements

- The system must process SRS files (PDF/DOCX) with 10–100 pages within ≤ 30 seconds.
- The response time from LLM APIs (Gemini/Groq) must not exceed 10 seconds per request.
- The system should support at least 10–20 concurrent users.
- The system should optimize chunking and API calls to reduce latency.

4.2. Usability Requirements

- The interface must be simple and easy to use:
 - Upload file
 - View results
 - Edit results
- Users do not need deep knowledge of Function Point Analysis.
- The dashboard must clearly display:
 - Function Points
 - Effort / Schedule
 - Cost
 - Breakdown (EI, EO, EQ, ILF, EIF)

4.3. Reliability Requirements

- The system must maintain uptime $\geq 95\%$.
- If the LLM API fails:
 - The system must display a clear error message
 - Allow users to retry
- No data should be lost during processing.

4.4. Security Requirements

- Only PDF/DOCX files are accepted.
- Uploaded SRS files must not be stored permanently.
- API keys (Gemini/Groq) must be secured in the backend.
- The system must validate files to prevent malicious uploads.

4.5. Maintainability Requirements

- The system must have clear modular structure:
 - Text Processing (NLTK)
 - AI Service
 - FP Calculation
- Easy to modify:
 - Prompt (few-shot examples)
 - AI model (Gemini, Groq, etc.)
- Logging must be implemented for debugging.

4.6. Scalability Requirements

- The system should scale to:
 - Larger SRS files
 - More concurrent users
- Possible improvements:
 - Queue for AI processing
 - Batch API requests

V. Use Case Specification

5.1. Actors

- User: Uploads SRS, views results, edits results, exports reports
- System: Handles processing, AI analysis, and calculations

5.2. Use Cases

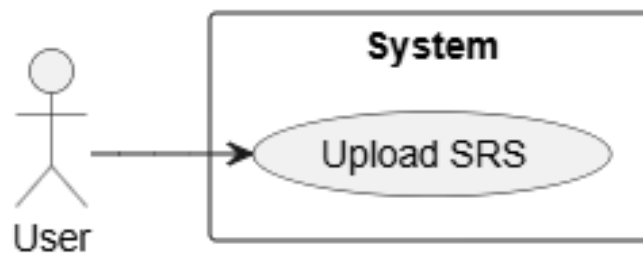
ID	Name
UC-01	Upload SRS
UC-02	Preprocess Text (NLP)
UC-03	Analyze with AI

ID	Name
UC-04	Calculate Function Points
UC-05	View Result
UC-06	Edit Result
UC-07	Export Report

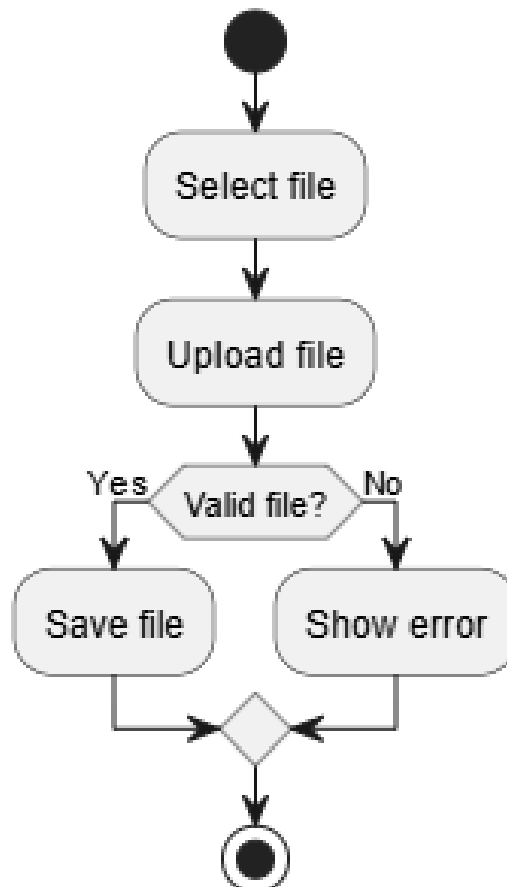
VI. UML DIAGRAMS

6.1. UC-01: Upload SRS

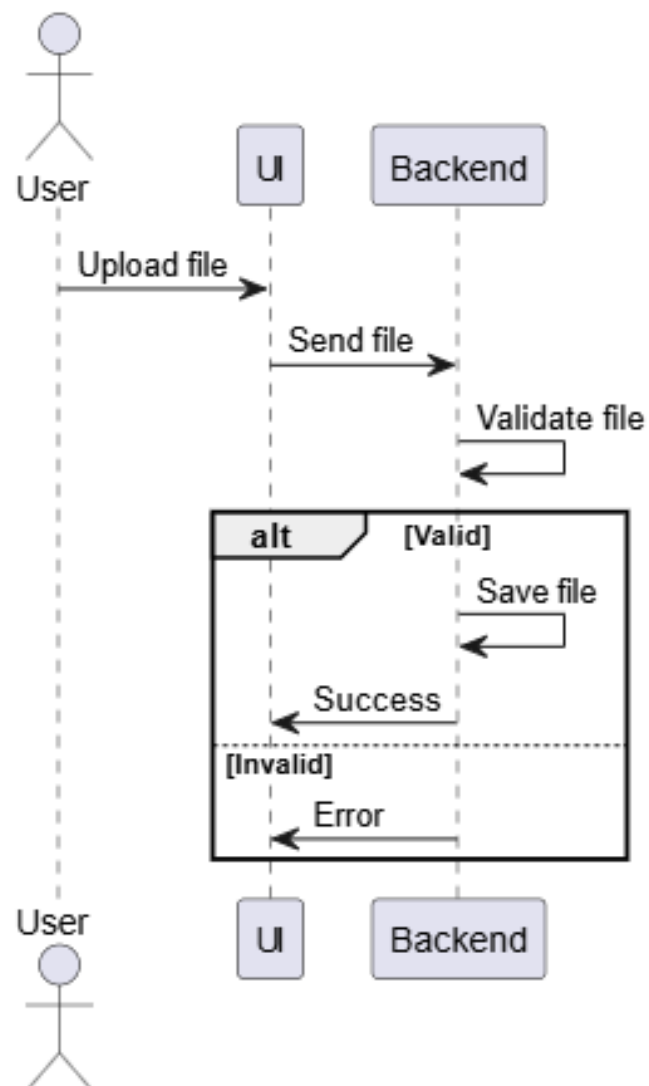
6.1.1. Use Case Diagram



6.1.2. Activity Diagram

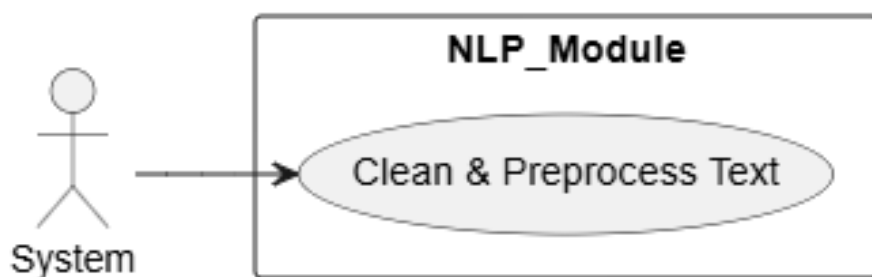


6.1.3. Sequence Diagram

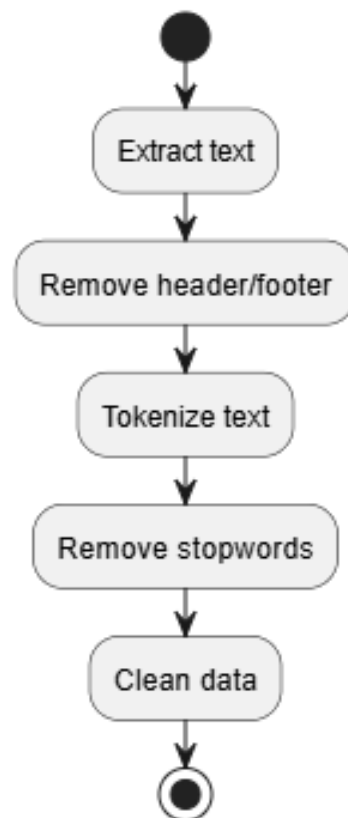


6.2. UC-02: Preprocess Text (NLP)

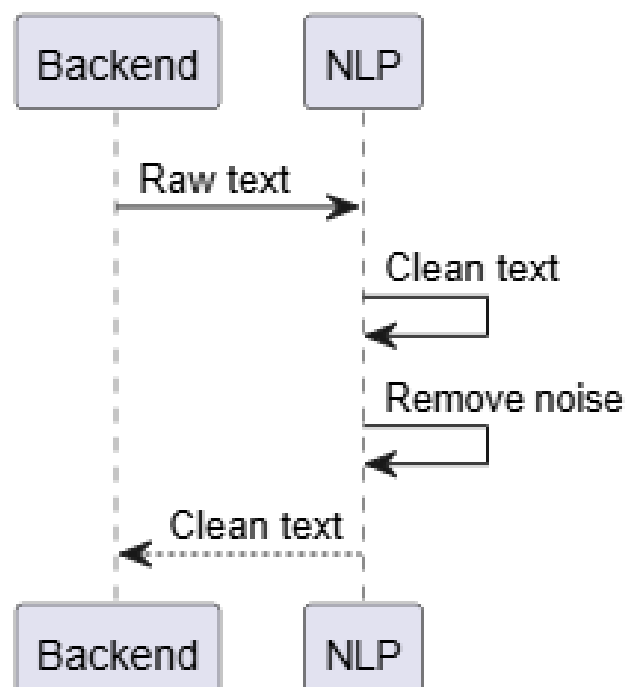
6.2.1. Use Case Diagram



6.2.2. Activity Diagram

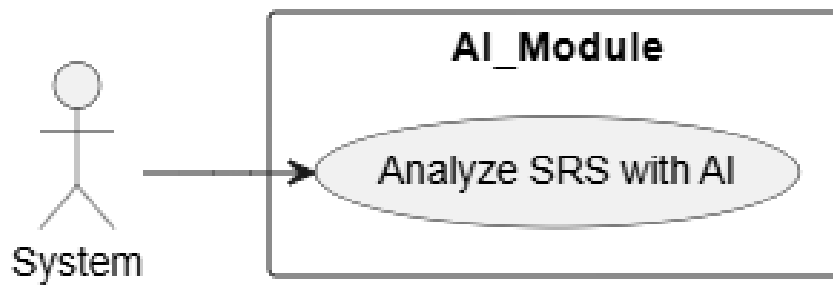


6.2.3. Sequence Diagram

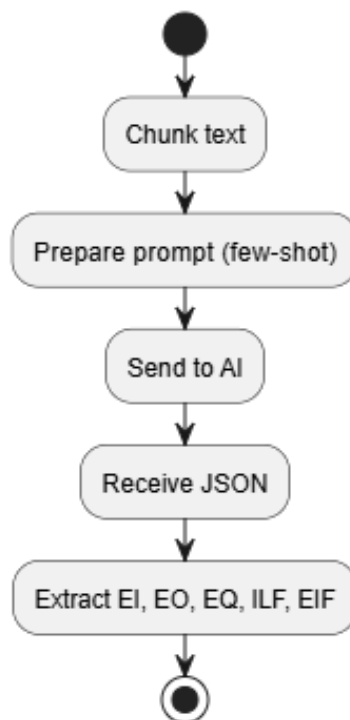


6.3. UC-03: Analyze with AI

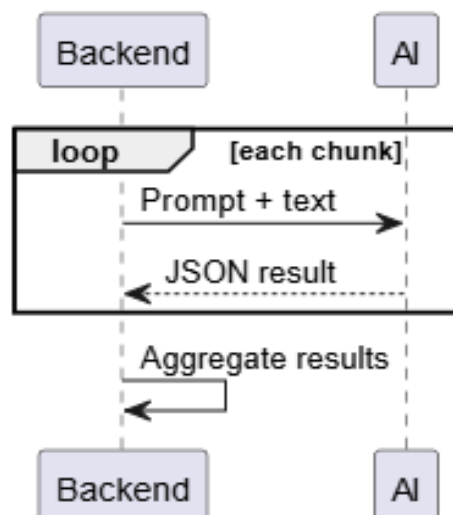
6.3.1. Use Case Diagram



6.3.2. Activity Diagram

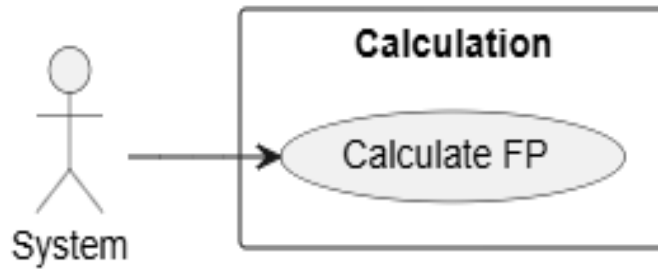


6.3.3. Sequence Diagram

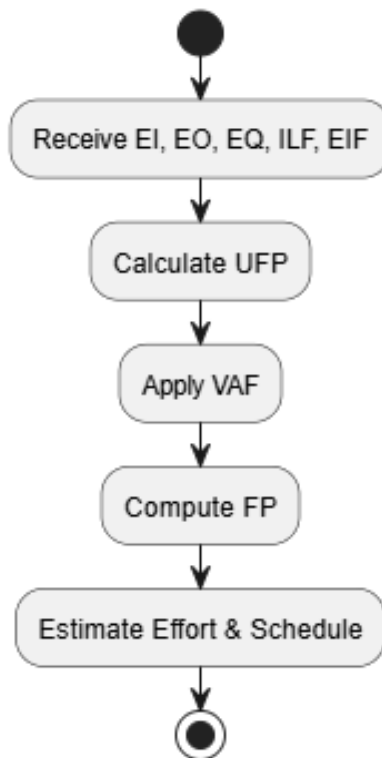


6.4. UC-04: Calculate Function Points

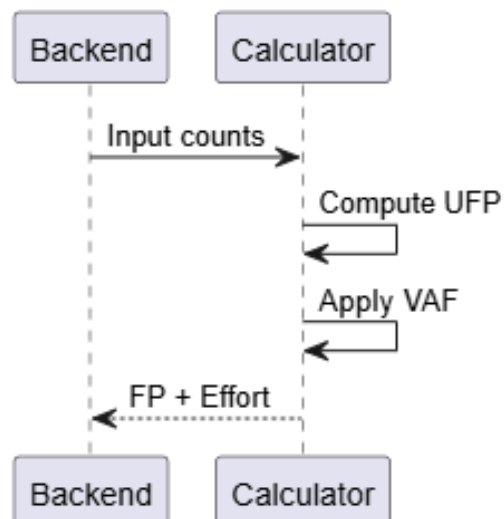
6.4.1. Use Case Diagram



6.4.2. Activity Diagram

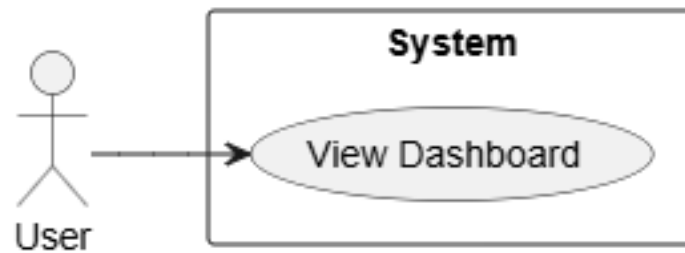


6.4.3. Sequence Diagram

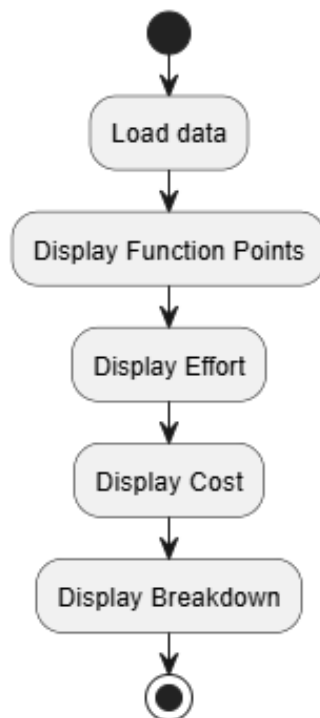


6.5. UC-05: View Result

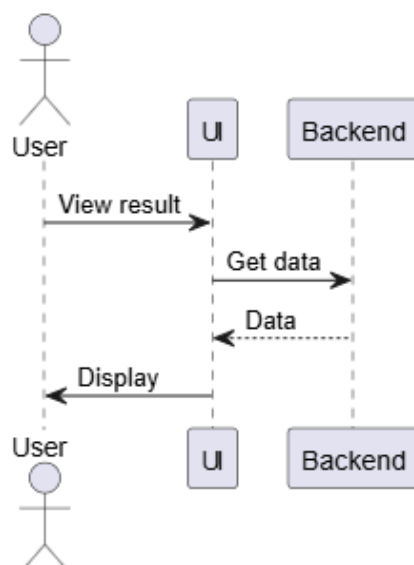
6.5.1. Use Case Diagram



6.5.2. Activity Diagram

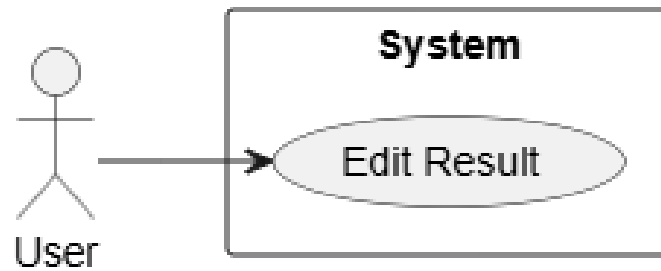


6.5.3. Sequence Diagram

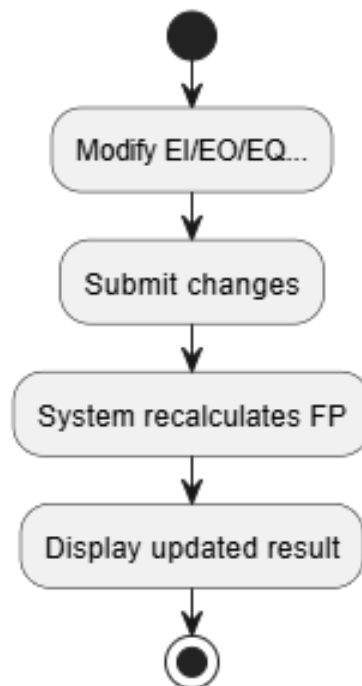


6.6. UC-06: Edit Result

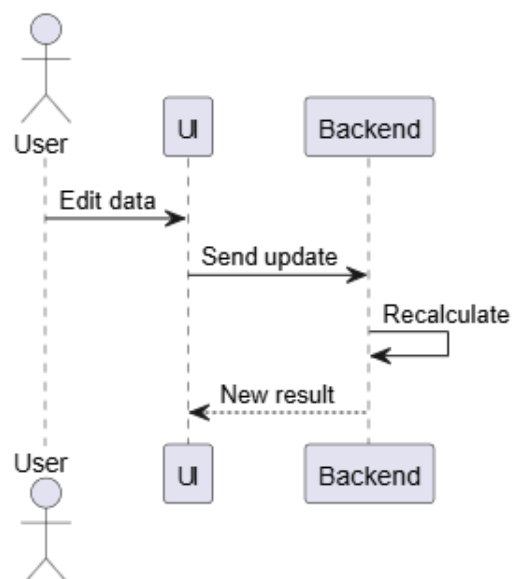
6.6.1. Use Case Diagram



6.6.2. Activity Diagram

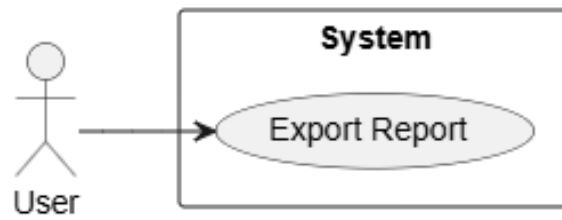


6.6.3. Sequence Diagram

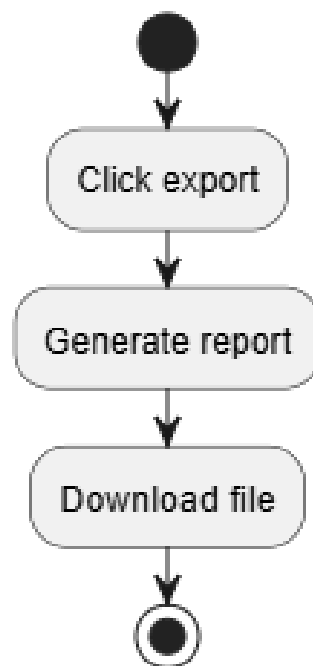


6.7. UC-07: Export Report

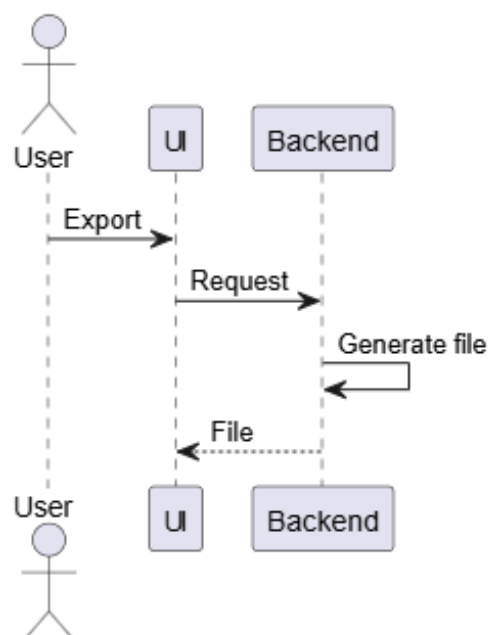
6.7.1. Use Case Diagram



6.7.2. Activity Diagram



6.7.3. Sequence Diagram



VII. Data Requirements

7.1. AI Output Data (AI Output JSON)

- Returned values must strictly be non-negative integers (Integer ≥ 0).

```
JSON
{
  "EI": int,
  "EO": int,
  "EQ": int,
  "ILF": int,
  "EIF": int
}
```

7.2. Final Estimation Data (Final Output)

- Estimated values must be floating-point numbers (Float ≥ 0), rounded to 2 decimal places.

```
JSON
{
  "FP": float,
  "Effort": float,
  "Schedule": float,
  "Cost": float
}
```

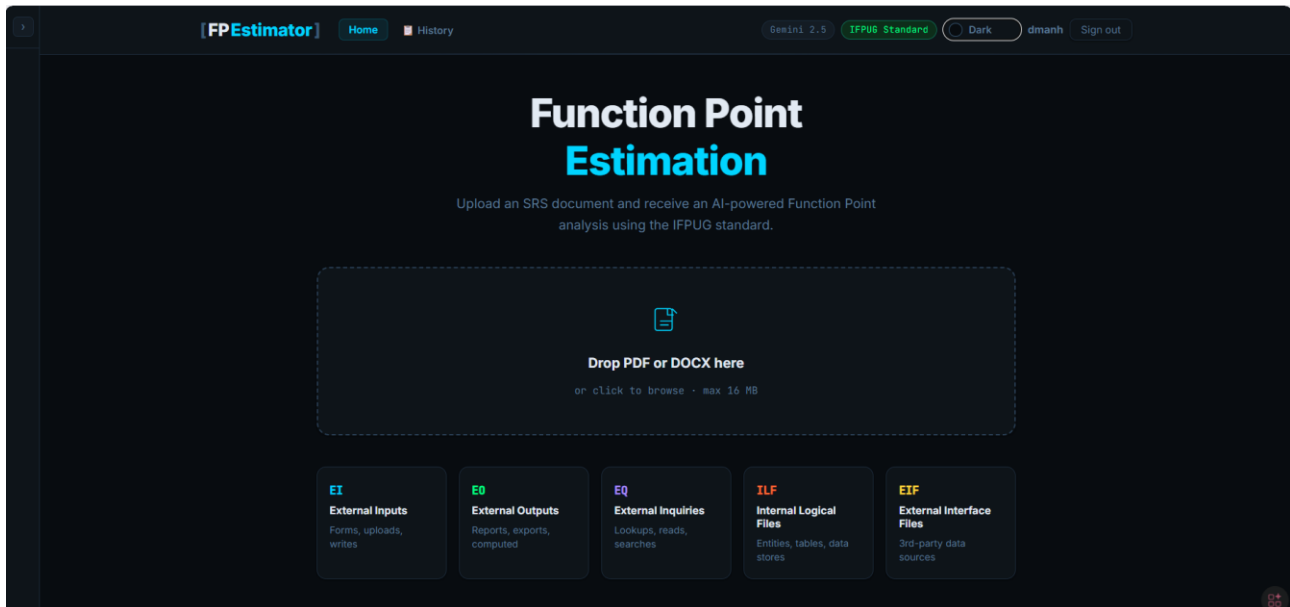
VIII. Acceptance Criteria

- Processing Performance:** The system must complete the entire process (from file upload to displaying results on the Dashboard) within ≤ 30 seconds for an SRS file with a size of ≤ 10 MB and a length of ≤ 50 pages.
- Analysis Accuracy:** The AI's accuracy in identifying components (EI, EO, EQ, ILF, EIF) must reach at least 75% when compared to manual extraction results by an expert on a sample test file.
- Dashboard Display:** Charts and metrics must render without layout breaks on common screen resolutions (from 1080p down to a 13-inch laptop screen). The data table must be smoothly editable without causing the browser to lag or freeze.

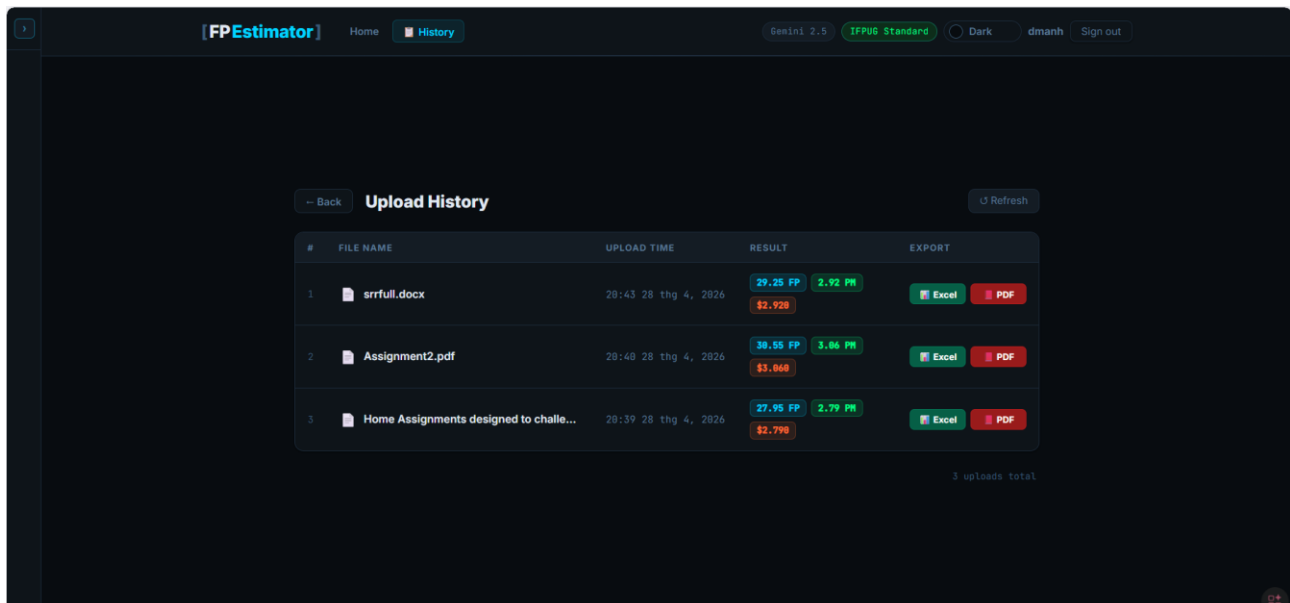
- Report Export Function: The .csv or .pdf file must download successfully, the data inside must be a 100% match with the final data displayed on the Dashboard, and it must fully support UTF-8 fonts (Vietnamese) without any character encoding errors.

IX. Demo Project

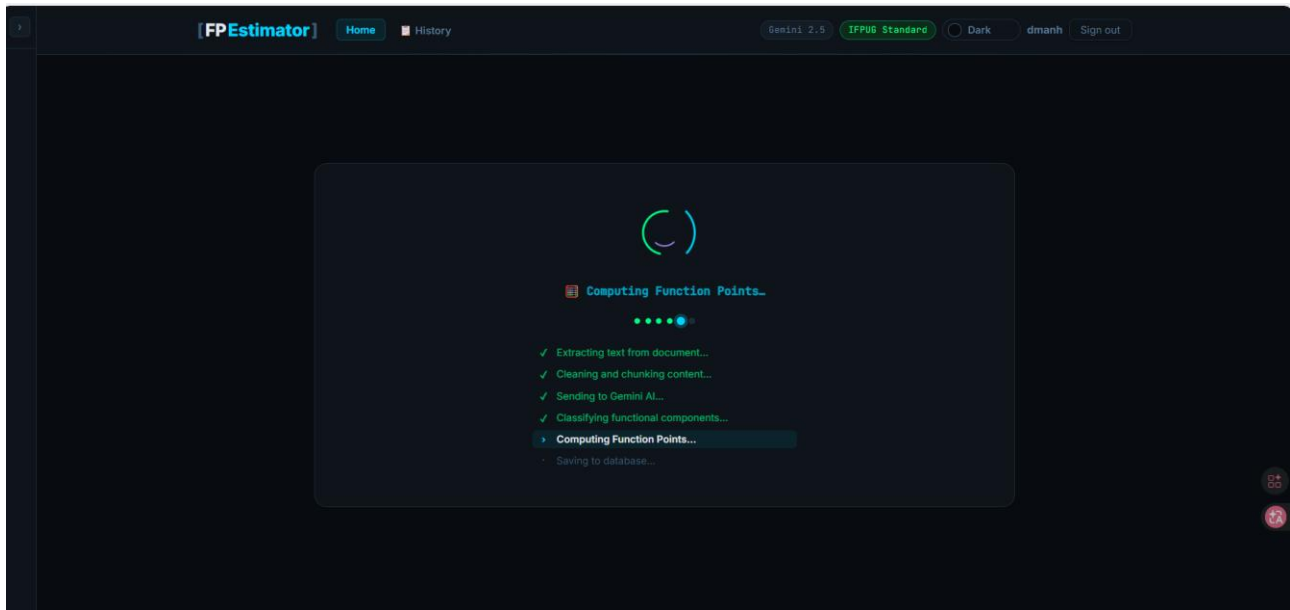
9.1. Dashboard



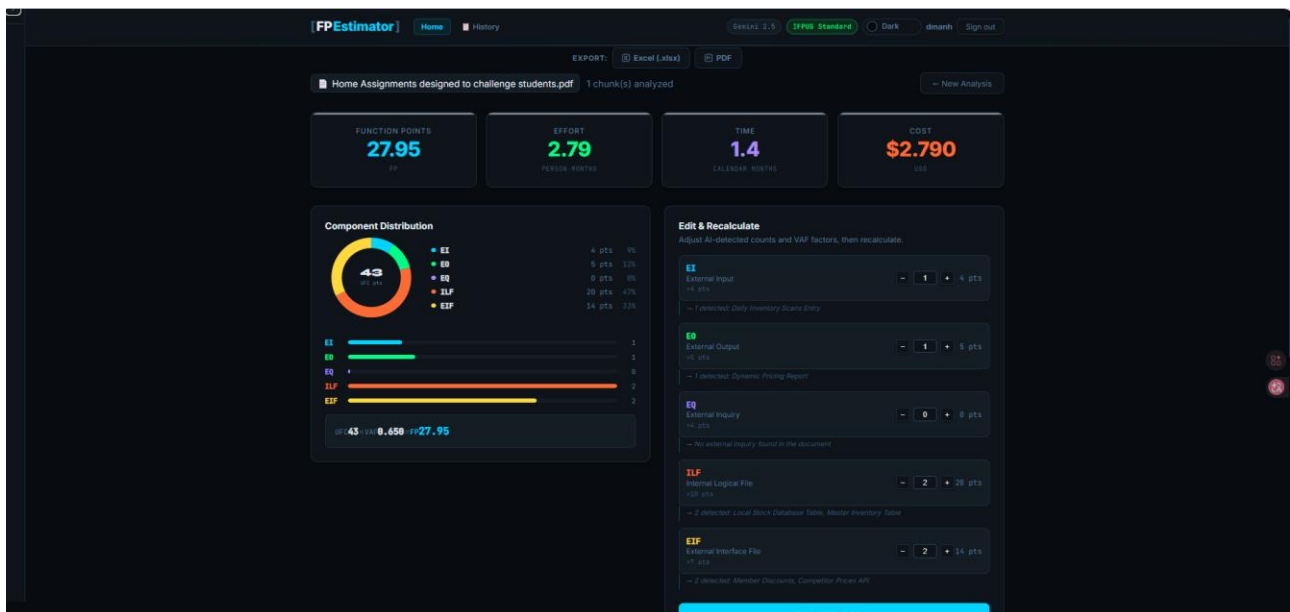
9.2. History



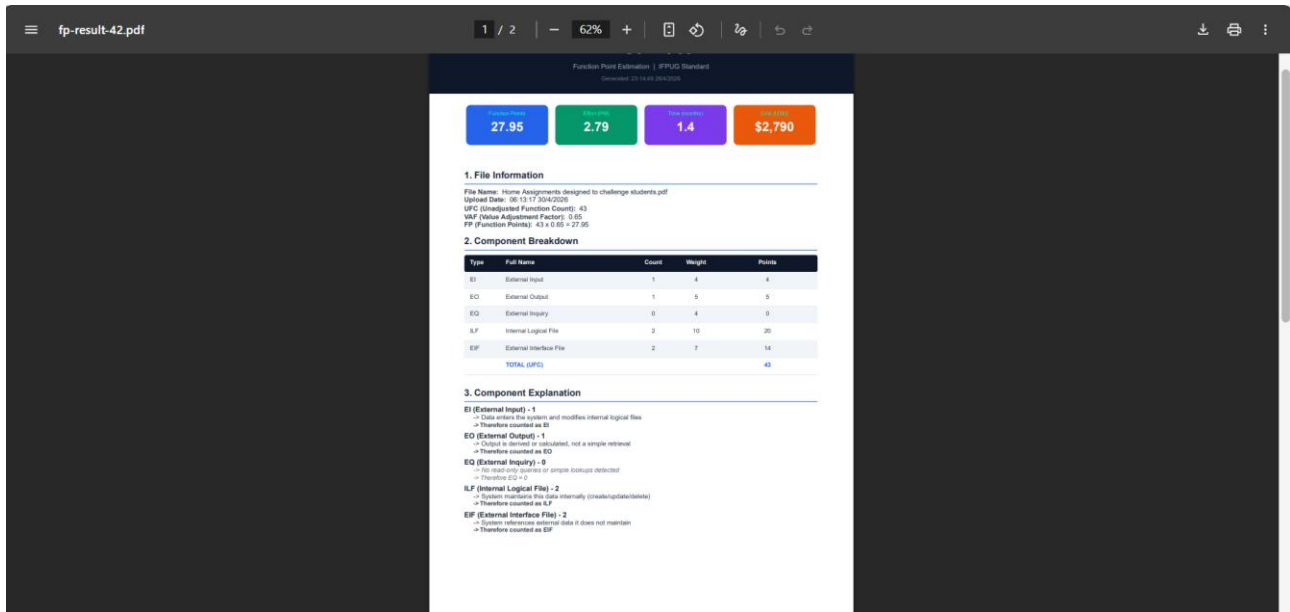
9.3. Loading when upload file



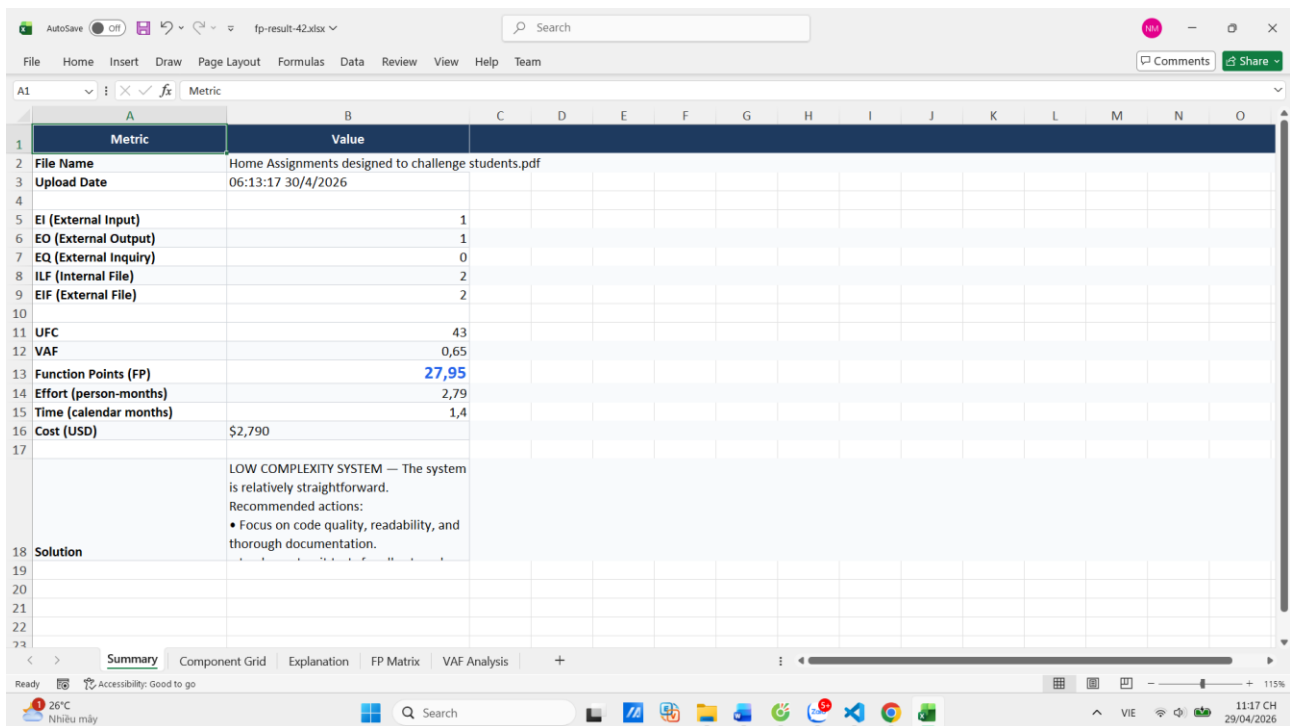
9.4. File analysis results



9.5. Export File PDF



9.5. Export File Exel



AI-Based Function Point Estimation Tool

Giải pháp tối ưu hóa quy trình ước lượng
phần mềm từ tài liệu SRS.

Nguyễn Văn Duy Mạnh

Lê Minh Trường Khả

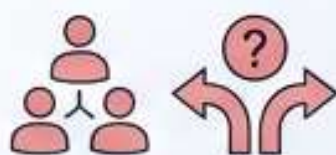
Trần Hoài Nam

⚠ Cách làm cũ ⚠



Chậm & Cồng Kênh

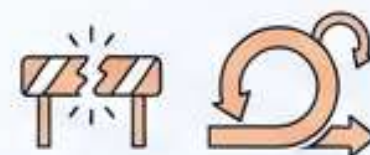
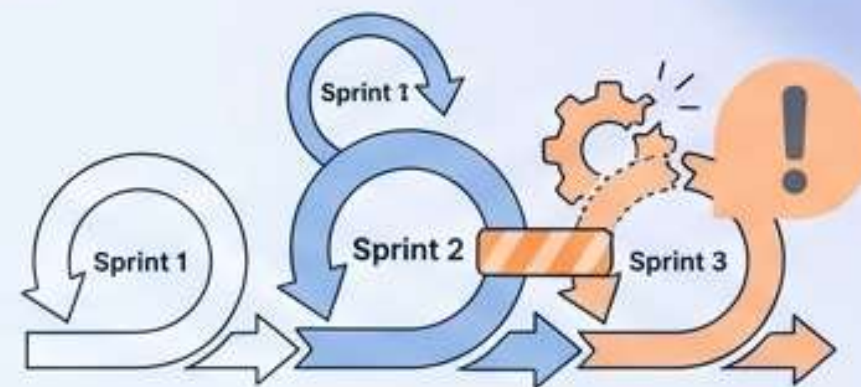
Đọc hàng trăm trang SRS & đánh dấu, đếm thủ công trên Excel làm lãng phí hàng giờ làm việc.



Thiếu Nhất Quán (Human Bias)

Con người bị ảnh hưởng bởi cảm tính -> Người A đếm là EO (tính toán), người B lại đếm là EQ (truy vấn đơn giản). Kết quả sai lệch.

Thực tế Agile



Khắc tinh của Agile

Thiếu linh hoạt: Yêu cầu dự án thay đổi liên tục mỗi Sprint. Đội ngũ không thể đếm lại FPA thủ công mỗi 2 tuần.



Tự động trích xuất

AI phân loại chuẩn xác 5 thành phần FPA (EI, EO, EQ, ILF, EIF) và tính toán VAF.



Tính toán toàn diện

Chuyển đổi tức thì sang Điểm Chức Năng (Total FPA), Nỗ lực (Effort / COCOMO), Lịch trình (Schedule), và Chi phí (Cost).



Layer 1 (Frontend)

React + Vite. Giao diện Web-based phản hồi tức thì.

Layer 2 (Backend Core)

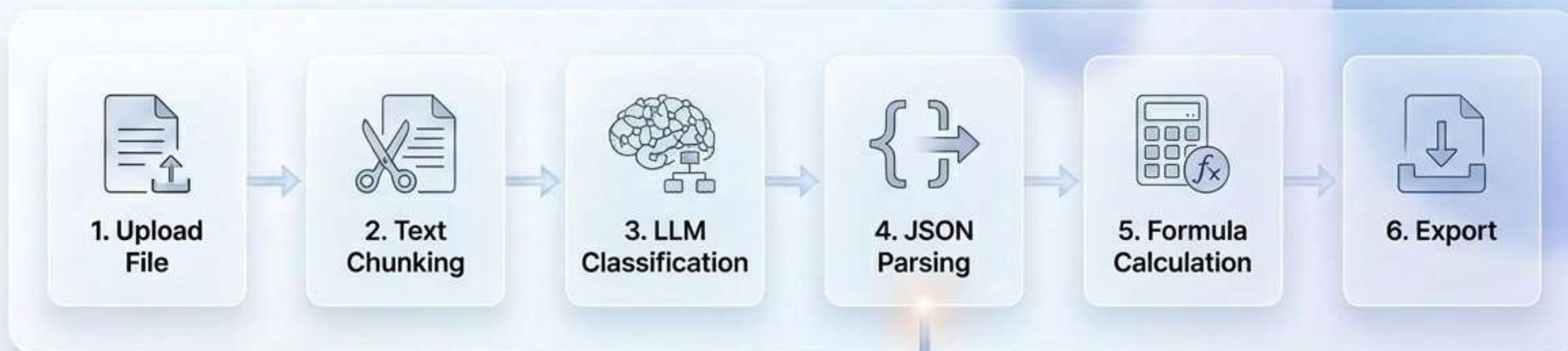
Python (Flask). Quản lý HTTP Requests, xử lý logic Tính toán UFC/VAF, và cấu hình API.

Layer 3 (NLP & AI Engine)

NLTK (Làm sạch & Cắt mảnh văn bản) kết nối API với LLMs (Gemini / Groq) kèm Prompt Engineering.

Layer 4 (Database)

MySQL để quản lý phiên và lưu trữ lịch sử.

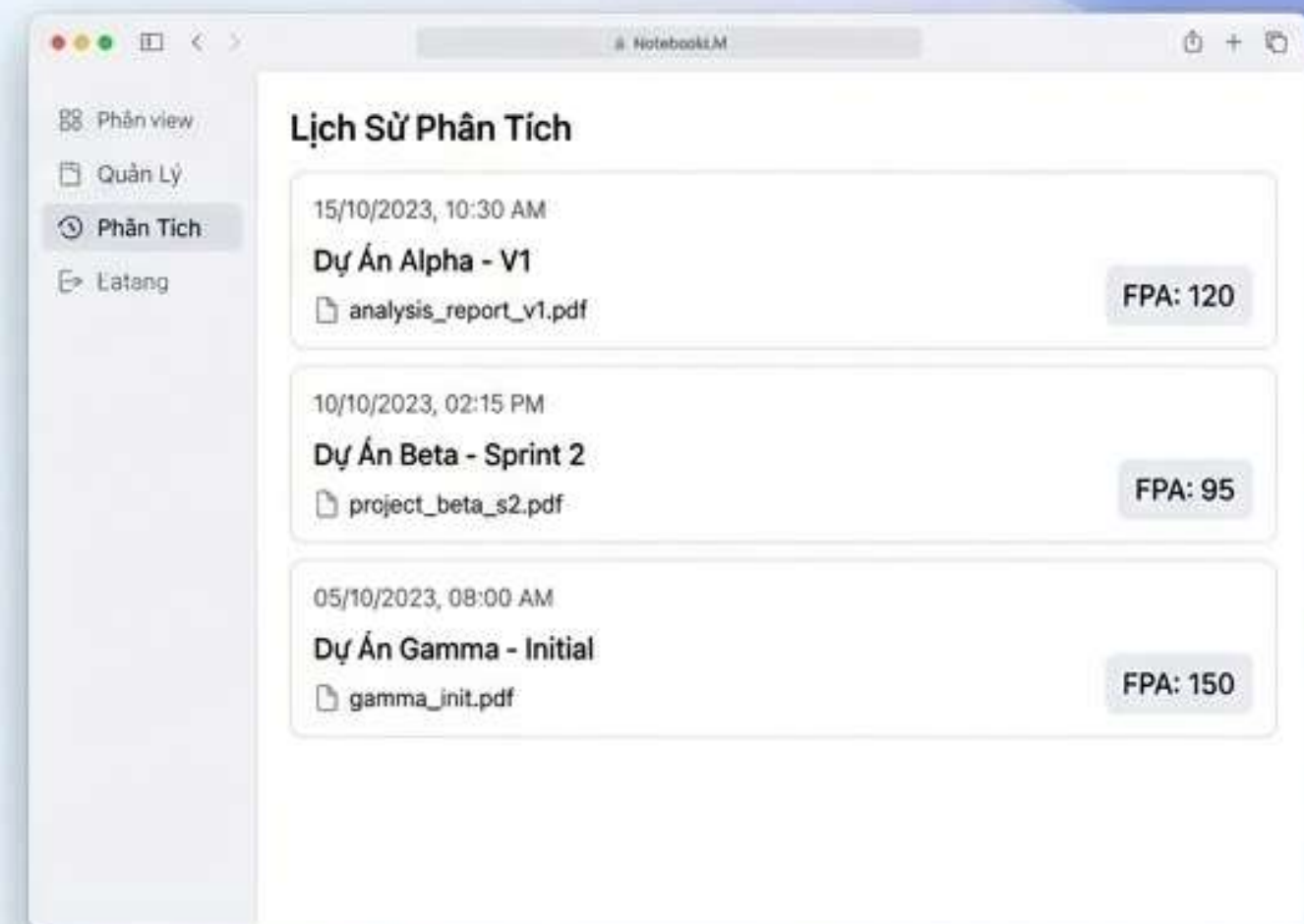


Hệ thống chỉ trích xuất các số nguyên dương không âm, đảm bảo tính ổn định tuyệt đối cho các công thức nhân ma trận trọng số phía sau.

```
{
  "EI": 12,
  "EO": 8,
  "EQ": 5,
  "ILF": 3,
  "EIF": 2
}
```

Tính năng Khác Biệt: Quản Lý Lịch Sử Phân Tích

- ✓ **Không chỉ là công cụ dùng một lần:** Mỗi tài liệu được lưu lại với kết quả cấu trúc hoàn chỉnh.
- ✓ **Truy xuất nhanh chóng:** Xem lại các bản phân tích cũ, đối chiếu thông số FPA trước và sau khi thay đổi yêu cầu.
- ✓ **Tối ưu cho Agile:** Phục vụ việc theo dõi nỗ lực mã hóa (Effort) qua các vòng lặp dự án (Sprints).



Hành Trình Người Dùng Không Ma Sát



Login:

Xác thực an toàn.



Upload:

Kéo thả
PDF/DOCX.



Processing:

Giao diện tự động
khóa (Lock UI)
-> Tránh spam
request.



Dashboard:

Hiển thị kết
quả tức thì.



Edit: Con người
trực tiếp tinh
chỉnh số liệu.

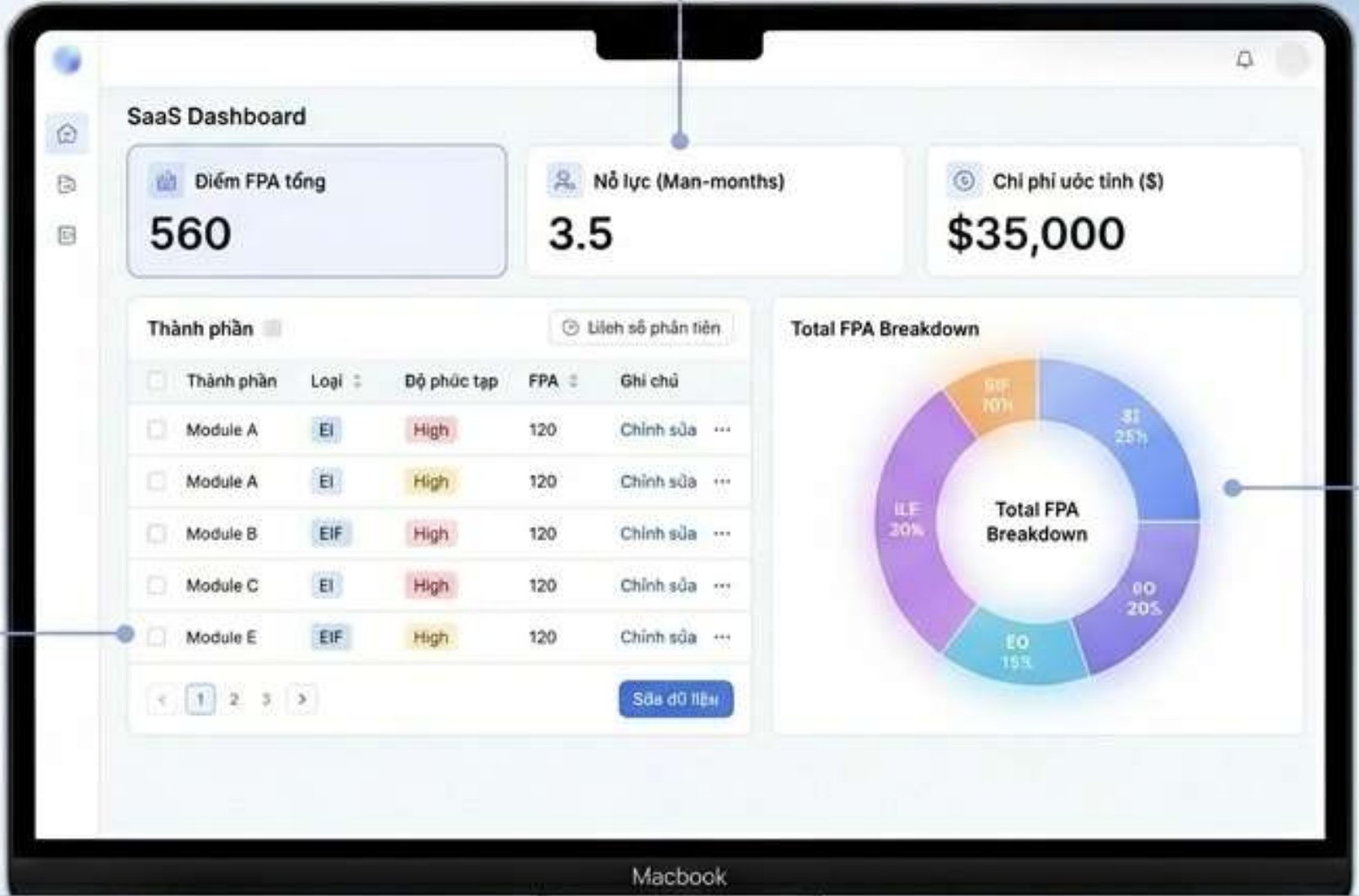


Save & Export:

Lưu lịch sử & tải
báo cáo.

SaaS Dashboard

Top KPIs: Điểm FPA tổng, Nỗ lực (Man-months), Chi phí ước tính (\$).



Human-in-the-loop
Table: Bảng chi tiết cho phép người dùng nhập/sửa dữ liệu và biểu đồ tự động cập nhật (Real-time render).

Visual Breakdown:
Biểu đồ Donut hiển thị tỷ trọng 5 thành phần (EI, EO, EQ, ILF, EIF).

Non-Functional Performance Metrics

<30s

Tốc Độ Siêu Tốc: Xử lý hoàn tất tài liệu SRS phức tạp (lên đến 50 trang / 10MB) dưới 30 giây.

$\geq 75\%$

Độ Chính Xác AI: Đạt độ chuẩn xác cao so với chuyên gia đo lường thủ công.

95%

Độ Tin Cậy Uptime: Hệ thống Auto-Retry thông minh khi API phản hồi lỗi.

Zero

Bảo Mật Tuyệt Đối: File nguồn bị xóa khỏi RAM ngay sau khi trích xuất. Không lưu trữ tài liệu mật.

AI Automation (Tốc độ)

Xóa bỏ quy trình đếm tay
tẻ nhạt, quét hàng nghìn
tờ trong vài giây.

Human-in-the-Loop (Độ chuẩn xác)

AI không thay thế con
người. Chuyên gia có
thể kiểm duyệt và ghi
đề kết quả trực tiếp.

Giải pháp Ước lượng Hoàn hảo

Agile Adaptability (Tính linh hoạt)

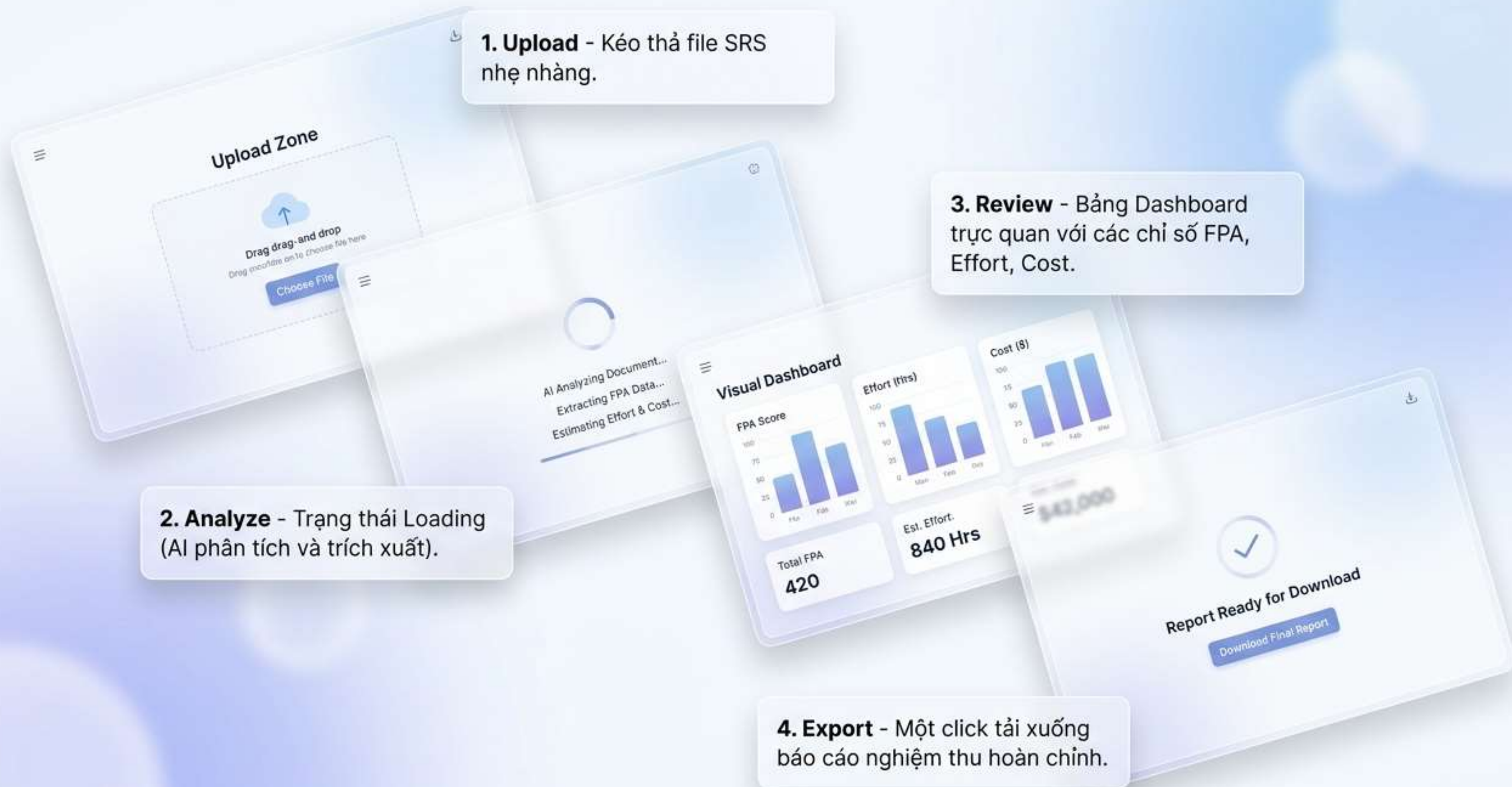
Sẵn sàng tái ước lượng chi phí
và thời gian ngay lập tức mỗi
khi Sprint thay đổi.

1. Upload - Kéo thả file SRS nhẹ nhàng.

2. Analyze - Trạng thái Loading (AI phân tích và trích xuất).

3. Review - Bảng Dashboard trực quan với các chỉ số FPA, Effort, Cost.

4. Export - Một click tải xuống báo cáo nghiệm thu hoàn chỉnh.



Biến quy trình ước lượng phức tạp thành trải nghiệm một chạm.



Nhanh hơn:

Tự động hóa hoàn toàn quy trình đọc hiểu.



Chính xác hơn:

Loại bỏ sai số do định kiến cá nhân.



Thực tiễn hơn:

Sẵn sàng áp dụng ngay cho môi trường phần mềm Agile.

Tương lai của việc đo lường phần mềm.

Thank you for listening